

Why Observational AI-Agent Pull Request Datasets Cannot Support Behavioral Inference: A Critical Analysis of Methodology Failures in Software Engineering Research

Ahmed Mursal

School of Computing

Edinburgh Napier University

Edinburgh, Scotland

40646515@live.napier.ac.uk

Abstract—We examine the suitability of observational pull request datasets for AI agent behavioral research using the AIDev dataset as a case study. Our analysis reveals fundamental validity failures that preclude meaningful inference: agent attribution exhibits obvious errors (GitHub Copilot: 379 users vs. millions in reality), statistical tests violate independence assumptions due to clustering, and measurement approaches show systematic bias. We demonstrate why large-scale observational studies of AI tools cannot support behavioral claims without controlled experimental designs. This methodology critique provides guidelines for valid AI tool evaluation and warns against common pitfalls in software engineering research that mistake dataset size for validity.

Index Terms—methodology, software engineering research, AI tools, empirical evaluation, validity threats

I. INTRODUCTION

The rapid adoption of AI-powered coding assistants has created an urgent need for empirical understanding of their impact on software development practices. Tools like GitHub Copilot, OpenAI Codex, Cursor, and Claude Code are now used by millions of developers, fundamentally changing how software is written, tested, and maintained. Understanding these tools' behavioral characteristics has become critical for evidence-based software engineering practice.

The software engineering research community has responded by analyzing large observational datasets scraped from development platforms. The AIDev dataset exemplifies this trend, containing 932,791 pull requests with agent attribution labels that researchers have used to draw behavioral conclusions about different AI tools. Studies using such datasets claim to identify "testing philosophies," "development patterns," and "behavioral signatures" across different agents.

However, such observational approaches face fundamental methodological challenges that compromise validity. This paper examines whether pull request datasets can support reliable behavioral inferences about AI tools. Through systematic analysis of the AIDev dataset, we identify critical failures in: (1) agent attribution reliability, (2) statistical assumption

compliance, and (3) measurement validity. These issues collectively demonstrate why observational PR data cannot support behavioral claims about AI tools.

A. The Stakes of Getting This Wrong

Invalid research conclusions about AI tool behavior can have serious consequences. Organizations may make tool selection decisions based on flawed empirical evidence. Developers may adopt practices based on spurious behavioral claims. The research community may build theoretical frameworks on unstable empirical foundations.

More broadly, the software engineering research community risks repeating historical mistakes where impressive-seeming large-scale studies later proved to be methodologically flawed. The appeal of "big data" can obscure fundamental validity threats, leading to confident conclusions based on unreliable evidence.

B. The Appeal of Large Datasets

Large observational datasets offer compelling advantages that make them attractive to researchers and reviewers alike. First, **massive sample sizes** provide impressive statistical power, making even small effects detectable and creating confidence in findings. The AIDev dataset's 932,791 observations dwarfs typical controlled studies, which rarely exceed thousands of participants.

Second, **real-world authenticity** appears to offer ecological validity that laboratory studies cannot match. Rather than artificial tasks in controlled settings, these datasets capture genuine development work by professional programmers in production environments.

Third, **comprehensive coverage** across multiple tools, languages, and organizational contexts suggests generalizability beyond narrow experimental conditions. The AIDev dataset spans 72,189 developers across diverse repositories, creating an impression of broad representativeness.

Finally, **resource efficiency** makes these studies attractive compared to expensive controlled experiments. Rather than

recruiting participants and designing experimental protocols, researchers can download datasets and begin analysis immediately.

However, this appeal creates a dangerous illusion of methodological rigor. Scale can mask fundamental validity threats, leading to confident conclusions based on systematically biased or unreliable data. The impressive sample size becomes a substitute for careful consideration of measurement validity, causal identification, and assumption compliance.

C. Research Questions

We address three methodological research questions:

RQ1: What validity threats prevent reliable inference about AI agent behavior from observational PR datasets? We systematically identify attribution failures, statistical violations, and measurement biases.

RQ2: How do these validity threats manifest in practice using the AIDev dataset? We provide concrete examples of each threat category with specific evidence.

RQ3: What methodological requirements are necessary for valid AI tool behavioral research? We specify design principles for reliable causal inference.

II. METHODOLOGY FAILURES IN OBSERVATIONAL AI RESEARCH

A. RQ1: Systematic Validity Threats

1) *Agent Attribution Unreliability*: Observational datasets typically derive agent labels through metadata extraction, commit message parsing, and self-reports. This approach suffers from multiple failure modes:

Detection Incompleteness. Automated attribution can only identify explicitly declared tool usage, missing silent assistance or tools used without obvious signatures.

Temporal Inconsistency. Tool adoption changes rapidly, but retrospective labeling may use contemporary usage patterns to infer historical tool use.

Mixed Authorship. Real development involves human editing of AI-generated code, making pure agent attribution impossible in observational settings.

Self-Report Bias. Developers may over-report or under-report tool usage based on perceived social desirability or organizational policies.

2) *Statistical Independence Violations*: Standard statistical tests assume independent observations, but PR datasets exhibit multiple clustering structures:

Repository Clustering. PRs within repositories share coding standards, review processes, and maintainer preferences. These systematic similarities violate independence.

Developer Clustering. Multiple PRs from the same developer reflect individual coding style, experience level, and tool preferences rather than independent observations.

Temporal Clustering. PRs within development cycles often follow sequential phases (feature development, testing, documentation) that create dependencies.

Project Clustering. PRs within the same project share requirements, constraints, and quality standards that influence testing practices.

These clustering violations invalidate chi-square tests, t-tests, and other standard procedures that assume independence, leading to inflated significance and misleading confidence intervals.

3) *Measurement Validity Problems*: Behavioral inference requires valid measurement instruments, but observational approaches suffer systematic bias:

Language Ecosystem Bias. Test detection methods favor popular languages and frameworks while systematically missing others, creating artificial behavioral differences.

Context Confusion. Mixing file paths, commit messages, PR descriptions, and code content creates inconsistent measurement criteria.

Quality vs. Quantity. Observational methods can only measure test presence, not test quality, coverage, or effectiveness.

Validation Inadequacy. Manual validation samples (typically 200-500 observations) cannot detect systematic biases across hundreds of thousands of observations spanning multiple domains.

B. RQ2: Evidence from the AIDev Dataset

1) *Attribution Failure Examples*: Examination of the AIDev dataset reveals obvious attribution errors that demonstrate label unreliability:

Impossible User Counts. GitHub Copilot appears with only 379 users despite having millions of actual subscribers. This 1000:1 under-detection indicates severe attribution failure.

Implausible Market Share. Lesser-known tools show more users than established market leaders, contradicting all external adoption data.

Extreme Single-User Cases. Devin shows 29,744 pull requests from a single user, suggesting bulk operations misattributed as agent behavior.

Temporal Inconsistencies. Tools show activity patterns that contradict known release timelines and API availability.

These examples demonstrate that agent labels cannot support group comparisons or behavioral inference.

2) *Statistical Assumption Violations*: The AIDev dataset exhibits severe clustering that invalidates standard statistical analysis:

Repository Concentration. Major agents are concentrated in small numbers of repositories, violating distributional assumptions.

Developer Concentration. Heavy users contribute hundreds of PRs each, creating extreme non-independence.

Project Dependencies. Sequential PRs within projects follow development workflows that create systematic patterns.

Standard chi-square analysis of such data produces meaningless p-values and inflated effect sizes.

3) *Measurement Bias Evidence*: Test detection approaches show systematic bias that undermines behavioral claims:

Keyword Bias. Detection favors Python/JavaScript terminology while missing Go, Rust, C++, and domain-specific testing approaches.

False Positive Patterns. Phrases like “tested in production,” “integration challenges,” and “test data” trigger detection without indicating actual tests.

Ecosystem Blindness. Languages with different testing conventions (property-based testing, contract testing, specification-driven development) are systematically under-detected.

C. RQ3: Requirements for Valid AI Tool Research

1) *Experimental Design Requirements:* Valid behavioral research about AI tools requires controlled experimental conditions that address the fundamental threats identified above:

Randomized Assignment. Participants must be randomly assigned to tools to eliminate selection bias and confounding. This prevents developers from choosing tools based on task characteristics, experience, or preferences that correlate with outcomes.

Controlled Tasks. Standardized programming tasks ensure fair comparison across conditions. Tasks should be representative of real development work while controlling for complexity, domain, and language requirements.

Verified Attribution. Direct tool instrumentation provides accurate usage measurement without inference. Rather than relying on metadata or self-reports, researchers should log actual tool interactions through IDE plugins or API monitoring.

Appropriate Controls. Baseline conditions without AI assistance enable causal attribution. Control groups help distinguish tool effects from general development practices.

Blinding When Possible. While complete blinding is difficult with AI tools, researchers can blind outcome assessment and use plausible attention controls.

Longitudinal Design. Within-subject designs where developers use multiple tools can control for individual differences in coding style and experience.

2) *Measurement Best Practices:* **Language-Stratified Validation.** Test detection must be validated within language ecosystems to avoid systematic bias.

Expert Review. Stratified manual validation across tools, languages, and project types with domain expertise.

Process Measurement. Focus on development processes and intermediate artifacts, not just final outcomes.

Quality Assessment. Evaluate test effectiveness through coverage analysis, mutation testing, or expert review.

3) *Statistical Approaches:* **Cluster-Robust Methods.** Use mixed-effects models, cluster-robust standard errors, or hierarchical approaches that account for non-independence.

Matched Sampling. Control confounders through propensity score matching, stratification, or covariate adjustment.

Effect Size Focus. Emphasize practical significance and confidence intervals over p-values.

Sensitivity Analysis. Test robustness to measurement assumptions and modeling choices.

III. A TAXONOMY OF VALIDITY THREATS IN OBSERVATIONAL AI TOOL RESEARCH

To systematize the methodological challenges we have identified, we propose a taxonomy of validity threats specific

to observational AI tool research. This framework can guide future researchers in identifying and addressing potential problems.

A. Attribution Validity Threats

Detection Bias. Systematic under-detection of certain tools due to methodological limitations creates artificial usage patterns.

Labeling Inconsistency. Different labeling approaches across repositories or time periods create heterogeneous attribution criteria.

Temporal Drift. Changes in tool adoption, naming conventions, or reporting practices over time create non-stationary attribution patterns.

Organizational Variation. Different corporate policies about tool disclosure create systematic differences in attribution completeness.

B. Measurement Validity Threats

Construct Underrepresentation. Measuring only easily observable phenomena (like test presence) while missing complex behavioral patterns.

Systematic Measurement Error. Consistent bias in measurement instruments that affects some groups differently than others.

Reactive Measurement. The possibility that developers change behavior when they know their activity is being tracked or analyzed.

Temporal Instability. Measurement instruments that work differently across time periods due to changing development practices.

C. Statistical Validity Threats

Assumption Violations. Using statistical procedures when their underlying assumptions are not met.

Multiple Comparisons. Conducting many statistical tests without appropriate corrections, inflating Type I error rates.

Effect Size Misinterpretation. Treating statistical significance as practical importance, especially with very large samples.

Confounding Structure. Unrecognized relationships between variables that create spurious associations.

IV. IMPLICATIONS FOR SOFTWARE ENGINEERING RESEARCH

A. The Illusion of Big Data Validity

Large sample sizes create false confidence in observational studies. The AIDev dataset’s 932,791 observations appear to provide unprecedented statistical power, but scale cannot overcome fundamental validity threats. In fact, larger samples can amplify systematic biases and create stronger false signals.

Sample Size $\neq$ Validity. Measurement error, attribution failures, and assumption violations affect large datasets just as severely as small ones. Scale may actually worsen these problems by making manual validation infeasible.

Significance Inflation. Massive samples make even tiny effects statistically significant, misleading researchers about practical importance.

Bias Amplification. Systematic errors compound with sample size, creating stronger false signals that appear more convincing.

B. Common Methodological Pitfalls

Software engineering researchers frequently fall into predictable traps when analyzing observational AI tool data:

Attribution Confidence. Trusting metadata-derived labels without validation leads to meaningless group comparisons.

Statistical Misapplication. Applying independence-assuming tests to clustered data produces invalid inference.

Causal Overreach. Interpreting correlational patterns as behavioral evidence without controlling for confounders.

Validation Theater. Using small manual validation samples to justify conclusions about massive datasets.

C. Guidelines for Valid Research

Before conducting observational AI tool research, researchers should:

1) Attribution Validation:

- 1) Verify tool labels match known adoption patterns
- 2) Investigate extreme distributions or single-user anomalies
- 3) Cross-validate against independent data sources
- 4) Report attribution uncertainty and sensitivity analyses

2) Statistical Diagnosis:

- 1) Quantify clustering through intracluster correlation
- 2) Test independence assumptions using diagnostic statistics
- 3) Use appropriate methods for clustered data
- 4) Report effective sample sizes, not just raw counts

3) Measurement Validation:

- 1) Stratify validation across relevant dimensions
- 2) Test for systematic bias in detection methods
- 3) Use domain experts for quality assessment
- 4) Report measurement uncertainty and limitations

V. RECOMMENDATIONS FOR FUTURE RESEARCH

A. Methodological Priorities

Controlled Experiments. The research community should prioritize randomized controlled trials over observational studies for behavioral claims about AI tools.

Within-Subject Designs. Longitudinal studies tracking individual developers across different tools can control for person-level confounders.

Mixed Methods. Combine quantitative measurement with qualitative investigation of decision-making processes.

Replication. Independent replication of tool evaluation findings across different contexts and populations.

B. Dataset Development

Verified Attribution. Future datasets should include direct tool instrumentation rather than inferential labeling.

Rich Context. Include repository characteristics, developer experience, project requirements, and organizational factors.

Quality Metrics. Extend beyond presence/absence to measure code quality, test effectiveness, and maintenance outcomes.

Longitudinal Structure. Track changes over time rather than cross-sectional snapshots.

VI. HISTORICAL PARALLELS AND LESSONS

The methodological challenges we identify in AI tool research echo historical problems in empirical software engineering and related fields. Understanding these parallels can help the community avoid repeating past mistakes.

A. The Replication Crisis in Psychology

Psychology faced a major replication crisis when researchers discovered that many published findings could not be reproduced. Key contributing factors included:

P-hacking. Researchers conducted multiple analyses until finding significant results, then reported only the significant findings.

Publication Bias. Journals preferentially published positive results, creating a distorted literature.

Weak Theory. Atheoretical data mining led to spurious discoveries that didn't replicate.

Methodological Flexibility. Researchers had too many degrees of freedom in analysis choices, enabling confirmation bias.

Software engineering research faces similar risks with large observational datasets. The temptation to mine data until finding "interesting" patterns, combined with publication incentives favoring positive results, can lead to unreliable findings.

B. Lessons from Medical Research

Medical research provides positive examples of how to conduct valid observational studies:

Preregistration. Researchers specify hypotheses and analysis plans before seeing data, preventing post-hoc rationalization.

Systematic Reviews. Meta-analyses combine evidence across multiple studies, reducing dependence on individual findings.

Reporting Standards. Standardized reporting guidelines (like STROBE for observational studies) ensure adequate methodological disclosure.

Causal Inference Methods. Sophisticated techniques like instrumental variables and regression discontinuity enable causal inference from observational data when properly applied.

C. Software Engineering’s Own History

Software engineering has its own examples of methodological evolution:

The Structured Programming Debates. Early claims about programming paradigm effectiveness were often based on weak empirical evidence, leading to prolonged debates.

Metrics Research. Many proposed software metrics showed statistical associations with quality measures but failed to demonstrate causal relationships or practical utility.

Process Improvement Studies. Claims about development process effectiveness often suffered from confounding, selection bias, and measurement problems similar to those we identify in AI tool research.

These historical examples suggest that the methodological challenges in AI tool research are not unique, but they are serious and require sustained attention.

D. Community Standards

The software engineering research community should establish methodological standards for AI tool evaluation:

Validity Checklists. Standardized criteria for evaluating observational AI tool studies.

Reporting Guidelines. Required disclosure of attribution methods, validation procedures, and assumption violations.

Peer Review Training. Education for reviewers on validity threats specific to AI tool research.

Negative Result Acceptance. Venues should welcome methodology critiques and null findings that advance research quality.

VII. LIMITATIONS AND SCOPE OF THIS ANALYSIS

This critique itself has limitations that readers should consider when interpreting our conclusions:

A. Scope Limitations

Single Dataset Focus. We examine only the AIDev dataset as our primary case study. Other observational datasets may exhibit different validity patterns, though we expect similar fundamental challenges.

Methodology Emphasis. We focus primarily on internal validity threats rather than exploring the practical utility of observational approaches for exploratory research or hypothesis generation.

Tool Evolution. We analyze data from 2021-2023, but AI tools evolve rapidly. Current tool behavior may differ significantly from the period covered by existing datasets.

B. Analytical Limitations

Alternative Approaches. We do not exhaustively explore all possible methods for addressing validity threats in observational settings. Advanced causal inference techniques might partially address some issues we identify.

Constructive Solutions. While we provide general guidance for valid research, we do not offer detailed protocols for specific study designs.

Cost-Benefit Analysis. We do not formally weigh the costs of controlled experiments against the benefits of observational studies for different research questions.

C. Broader Context

Generalizability. Our findings about the AIDev dataset may not generalize to all observational software engineering research, though we believe the fundamental principles apply broadly.

Research Ecosystem. We focus on methodological issues rather than broader questions about research incentives, publication practices, or community standards that influence study quality.

Practical Constraints. We acknowledge that controlled experiments are not always feasible, and observational studies may be the only available approach for some research questions.

VIII. FUTURE DIRECTIONS FOR VALID AI TOOL RESEARCH

Based on our analysis of methodological failures in observational AI tool research, we outline promising directions for future work that can address the validity threats we have identified.

A. Methodological Innovations

Hybrid Designs. Combining controlled experiments with observational data can leverage the strengths of both approaches. For example, researchers could conduct randomized trials to establish causal relationships, then use observational data to test generalizability.

Natural Experiments. When full experimental control is impossible, researchers can look for natural experiments where tool assignment is quasi-random due to organizational policies, timing, or other factors.

Instrumental Variables. Economic techniques like instrumental variables could help identify causal effects from observational data when valid instruments are available.

Regression Discontinuity. When tools are assigned based on threshold rules (e.g., organization size, experience level), regression discontinuity designs can enable causal inference.

B. Data Infrastructure

Verified Attribution Systems. The research community should develop standardized systems for accurately tracking tool usage through IDE instrumentation, API monitoring, or other direct measurement approaches.

Longitudinal Cohorts. Following the same developers over time as they adopt different tools can control for individual differences and enable within-subject comparisons.

Rich Context Data. Future datasets should include detailed information about projects, organizations, tasks, and developer characteristics to enable proper statistical control.

Quality Metrics. Moving beyond simple presence/absence measures to assess code quality, test effectiveness, and long-term maintenance outcomes.